

模 型 调 优

时间：2025年5月

TRANSWARP
星 环 科 技



目录 >

CONTENTS

- 1 模型调参
- 2 正则化
- 3 不平衡学习



1 chapter

模型调参与优化

模型调参概述

- 在机器学习中，我们经常会听到“**参数**”这个词，有时候需要结合上下文才能准确理解当前说的这个参数指的是什么。因为模型有两个参数：

- 一个指的是模型的**权重值**
- 一个指的是模型的**超参数**

调参就是**调超参数**。



模型**权重**指的是模型训练完成之后得到的结果，例如线性回归模型的**系数**($w_1, w_2 \dots w_n, b$)，决策树模型的**分割参数**等。

模型**超参数**指的是模型训练的时候算法要设置的一些**选项**，例如逻辑回归模型训练的时候需要指定：使用**L1**还是**L2**正则化？正则化强度要多大？对应算法中就是 **penalty** 和 **C** 这两个参数。

模型调参概述

- 在每一个算法都有各自对应的超参数

IO参数 **算子参数** 高级参数

判别阈值	0.5
最大迭代次数	10
正则化系数	0.0
迭代阈值	1.0E-6
弹性网络参数	0.0
☒ 标准化参数	
☒ 截距项参数	

IO参数 **算子参数** 高级参数

最大迭代次数	100
正则化系数	
阈值	0.5
☒ 标准化	
迭代阈值	1.0E-6
☒ 是否使用截距	

IO参数 **算子参数** 高级参数

K近邻	5
随机树种子	5

IO参数 **算子参数** 高级参数

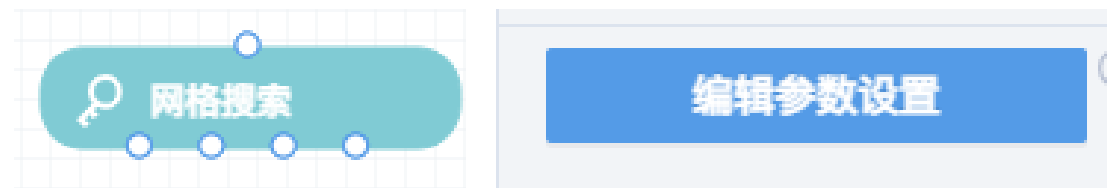
树的深度	5
最大分桶数	32
最小分割大小	1
最小增益	0.0
不纯度	entropy
种子	5

如何调参？

使用**可靠的评估方法**进行**穷举搜索**并返回效果**最好**的一组参数



保证评估的**可靠性**



网格搜索 (Grid Search)

- **网格搜索**是一项模型**超参数优化技术**，常用于优化**三个或者更少数量**的超参数，本质是一种**穷举法**
- 例如，我们三个需要优化的**超参数A,B,C**，候选的取值分别是{1,2}, {3,4}, {5,6}。则所有可能的参数取值**组合**组成了一个8个点的3维空间网格如下：
$$\{ (1,3,5), (1,3,6), (1,4,5), (1,4,6), (2,3,5), (2,3,6), (2,4,5), (2,4,6) \}$$
- 网格搜索就是通过**遍历**这8个可能的参数取值组合，进行**训练和验证**，最终得到**最优解**。

缺点

耗时；参数越**多**，候选值越**多**，耗费时间越**长**！所以，一般情况下，先定一个**大范围**，然后**再细化**。

导入工具包，数据处理

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
```

```
# 获取数据集，并将数据集划分为训练集和测试集
```

```
# 1. 获取数据集
```

```
iris = load_iris()
```

```
# 2. 数据基本处理 -- 划分数据集
```

```
x_train, x_test, y_train, y_test = train_test_split(iris.data, iris.target, random_state=22)
```

```
# 3. 特征工程：标准化
```

```
# 实例化一个转换器类
```

```
transfer = StandardScaler()
```

```
# 调用fit_transform
```

```
x_train = transfer.fit_transform(x_train)
```

```
x_test = transfer.transform(x_test)
```

模型训练

```
# 训练模型，并输出最有参数
#实例化 knn
knn=KNeighborsClassifier()
#指定参数字典
param_dict={'n_neighbors':[3,5,7,9]}
#实例化Gridsearchcv
gs=GridSearchCV(knn, param_grid=param_dict, cv=3)
#训练
gs.fit(x_train, y_train)
#输出最有参数
gs.best_params_
```

```
{'n_neighbors': 5}
```

#重新训练模型

```
knn=KNeighborsClassifier(n_neighbors=5)
knn.fit(x_train, y_train)
```

```
KNeighborsClassifier(algorithm='auto', leaf_size=30, metric='minkowski',
                      metric_params=None, n_jobs=None, n_neighbors=5, p=2,
                      weights='uniform')
```

参数输出

```
# 分别输出训练集和测试集的knn值  
print("训练集:", knn.score(x_train, y_train))  
print("测试集:", knn.score(x_test, y_test))
```

```
训练集: 0.9732142857142857  
测试集: 0.9473684210526315
```

```
# 输出最优参数  
gs.best_estimator_
```

```
KNeighborsClassifier(algorithm='auto', leaf_size=30, metric='minkowski',  
                      metric_params=None, n_jobs=None, n_neighbors=5, p=2,  
                      weights='uniform')
```



1 2 chapter

正则化

模型调优：正则化

- **正则化 (Regularization)** 是机器学习中一种常用的技术，其主要目的是**控制模型复杂度**，**减小过拟合**。最基本的正则化方法是在**原目标（代价）函数** 中添加**惩罚项**，对复杂度高的模型进行“**惩罚**”。

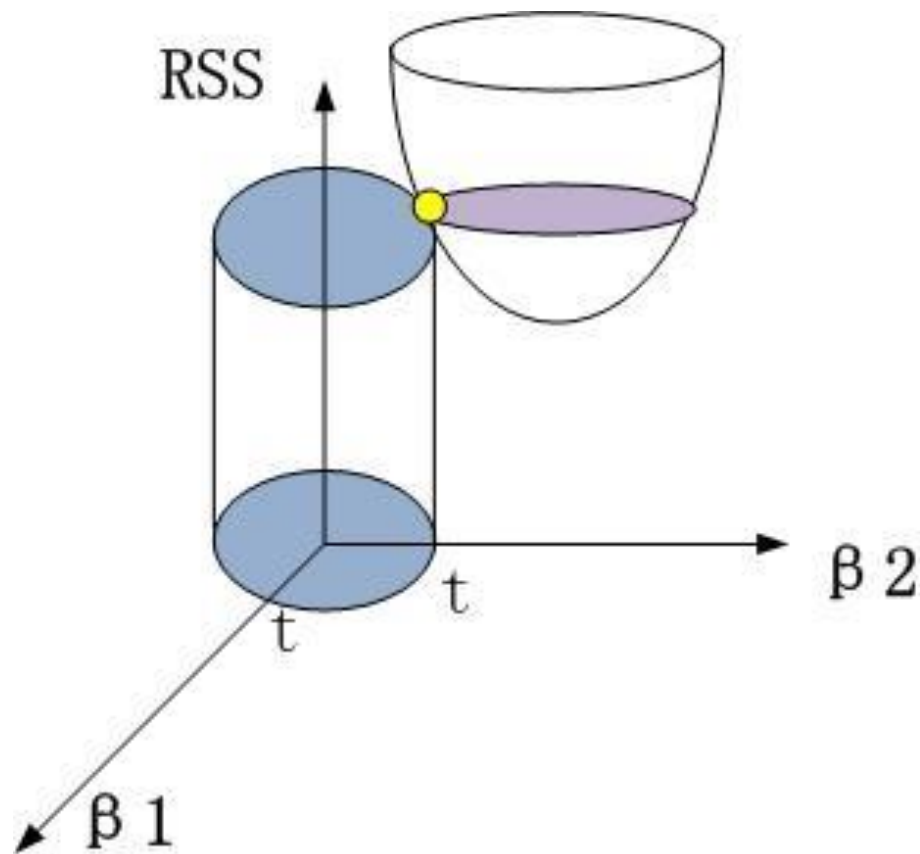
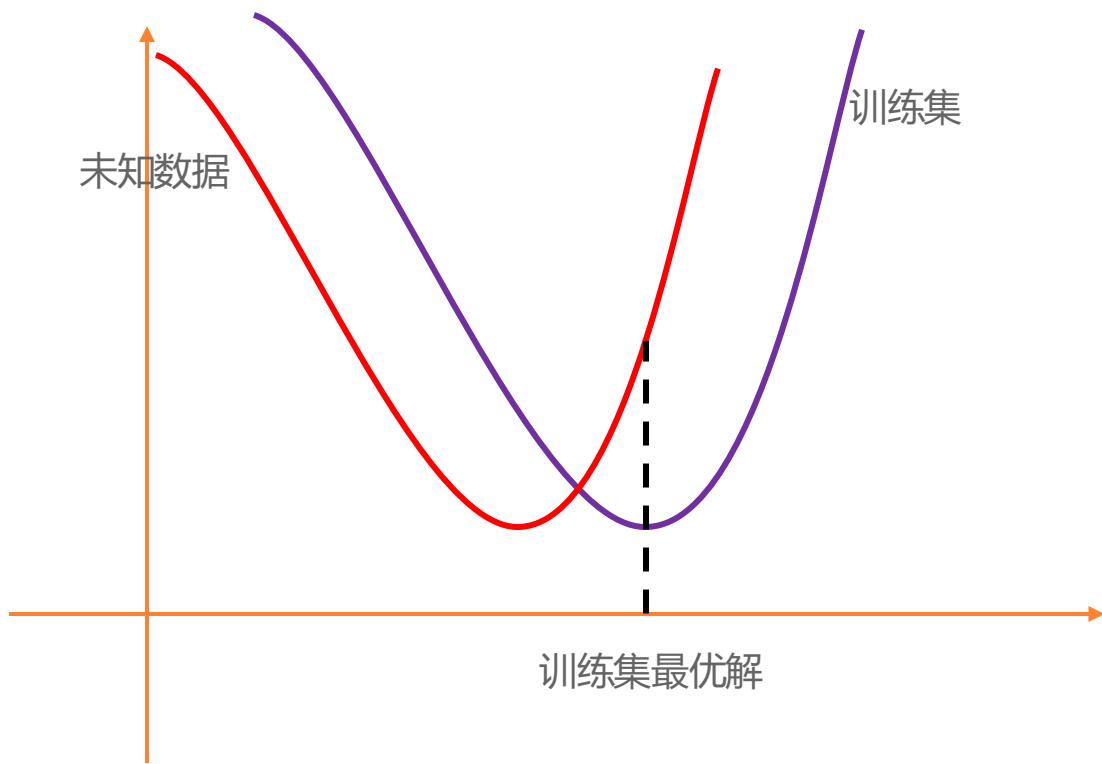
$$\tilde{J}(w; X, y) = J(w; X, y) + \alpha \Omega(w)$$

Diagram illustrating the components of the regularized cost function equation:

- $\tilde{J}(w; X, y)$ points to: 惩罚后的代价函数
- $J(w; X, y)$ points to: 原代价函数
- α points to: 惩罚项系数, 调整惩罚力度
- $\Omega(w)$ points to: 惩罚项

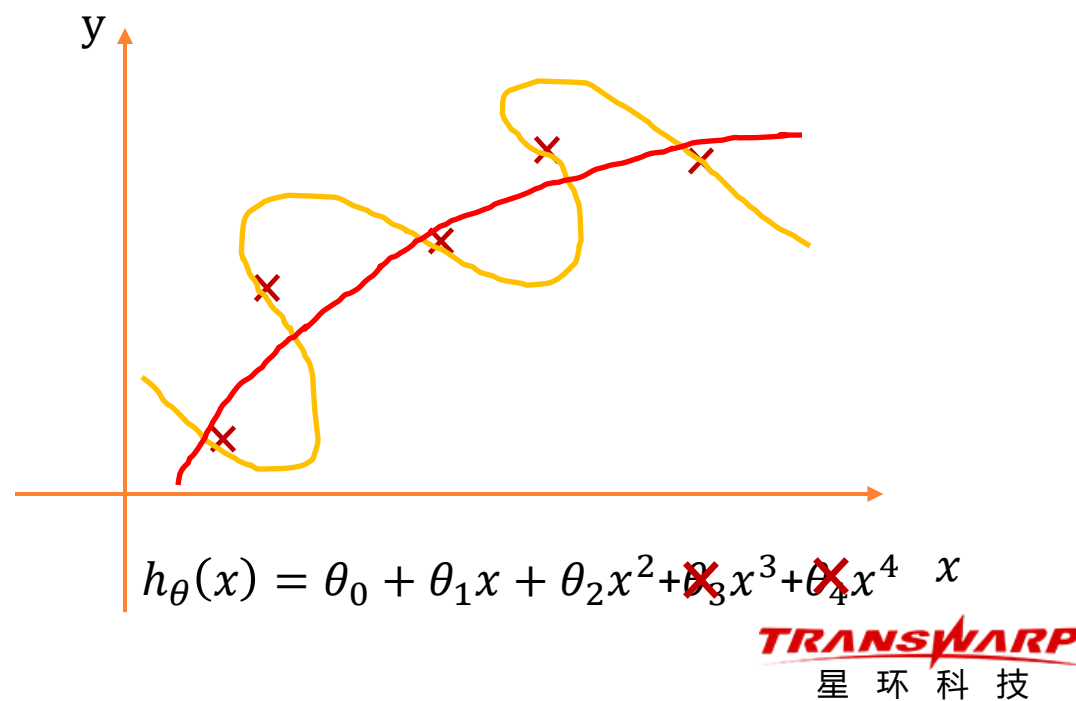
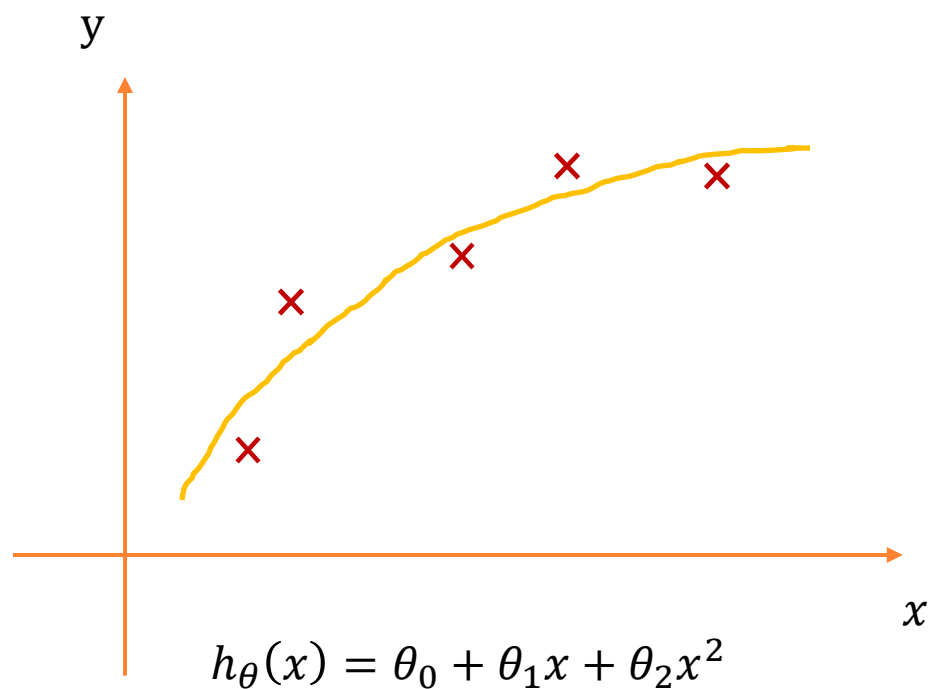
模型调优：正则化

- 正则化本质：抑制模型在训练集上求最优解
- 训练集的最优解不是未知数据的最优解



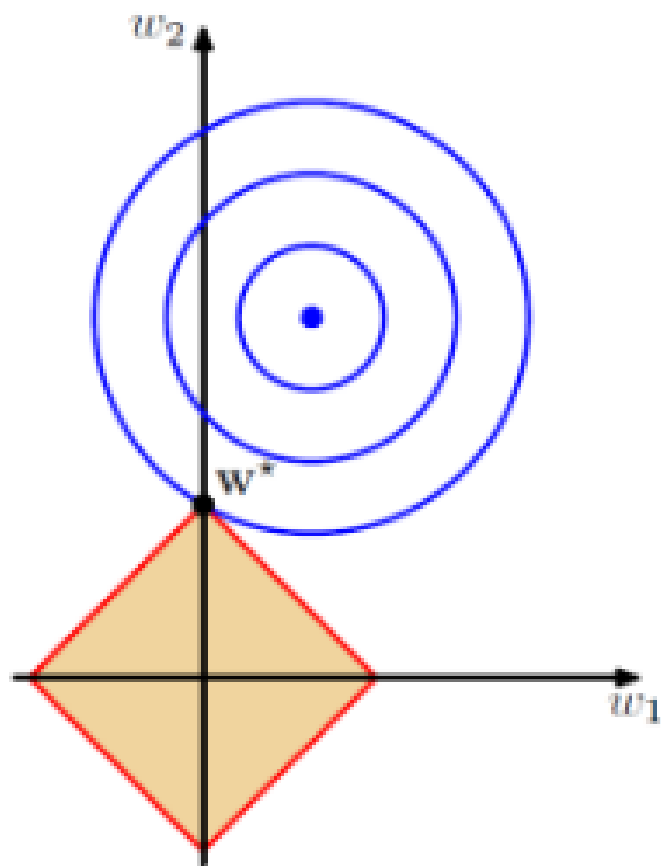
模型调优：正则化

- 正则化本质：抑制模型在训练集上求最优解
- 训练集的最优解不是未知数据的最优解



模型调优：正则化种类

■ L1 正则化



$$\tilde{J}(w; X, y) = J(w; X, y) + \alpha \Omega(w)$$

$$l_1 : \Omega(w) = \|w\|_1 = \sum_{i=1}^k |w_i|$$

L1正则化会产生**稀疏模型**（模型权重w会有很多为0）

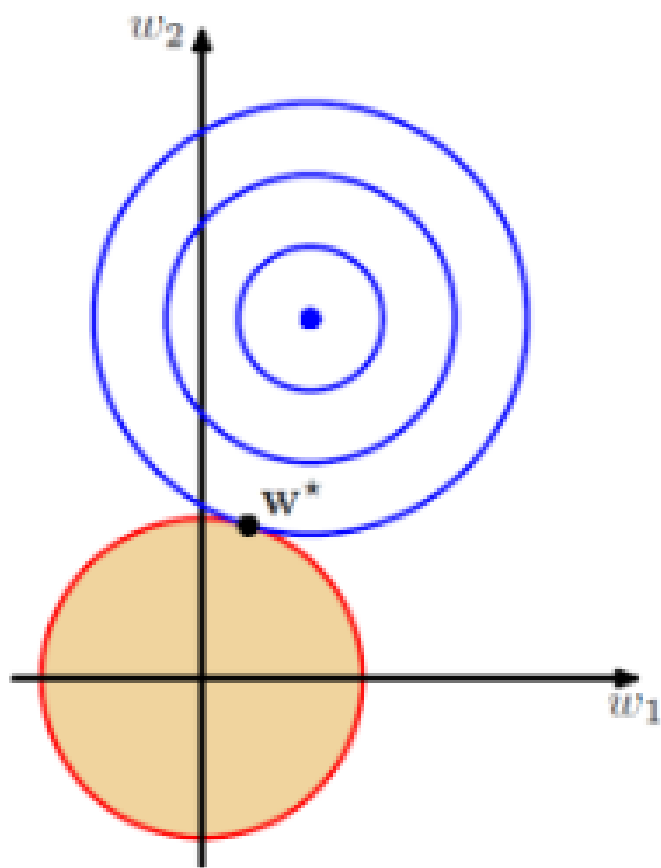
L1正则化假设权重w的先验分布为**拉普拉斯分布**

带有L1正则化的回归模型通常被称为**拉索回归Lasso Regression**



模型调优：正则化种类

■ L2 正则化



$$\tilde{J}(w; X, y) = J(w; X, y) + \alpha \Omega(w)$$

$$l_2 : \Omega(w) = \|w\|_2 = \sum_{i=1}^k w_i^2$$

L2正则化会让**权重w接近0但不为0**

L2正则化假设权重w的先验分布为**高斯分布**

带有L2正则化的回归模型通常被称为**岭回归Ridge Regression**

模型调优：正则化种类

- 弹性网络 (L1和L2的线性组合)
- 弹性网络是一种使用 L1, L2 范数作为先验正则项训练的线性回归模型。
 - $\rho=1$:退化为L1正则
 - $\rho=0$:退化为L2正则

$$\tilde{J}(w; X, y) = J(w; X, y) + \alpha\Omega(w)$$

$$\alpha\rho\|w\|_1 + \frac{\alpha(1-\rho)}{2}\|w\|_2^2$$

导入工具包，处理数据

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge, RidgeCV, Lasso, LinearRegression, ElasticNet
from sklearn.datasets import load_boston
from sklearn.metrics import mean_squared_error
```

1. 获取数据

```
data = load_boston()
```

2. 数据集划分

```
x_train, x_test, y_train, y_test = train_test_split(data.data, data.target, random_state=22)
```

3. 特征工程-标准化

```
transfer = StandardScaler()
```

```
_train = transfer.fit_transform(x_train)
```

```
x_test = transfer.transform(x_test)
```

训练模型

```
# 模型训练并计算误差均方误差  
lr=LinearRegression()  
lr.fit(x_train,y_train)  
y_pred=lr.predict(x_test)  
mean_squared_error(y_test,y_pred)
```

272.2072608827859

岭回归与拉索回归

```
# L2正则化 (岭回归)
```

```
ridge=Ridge(alpha=100)
```

```
ridge.fit(x_train,y_train)
```

```
Ridge(alpha=100, copy_X=True, fit_intercept=True, max_iter=None,  
      normalize=False, random_state=None, solver='auto', tol=0.001)
```

```
# 计算正则化后的MES
```

```
#评估
```

```
y_pred=ridge.predict(x_test)
```

```
mean_squared_error(y_test,y_pred)
```

```
266.6695244000735
```

```
# L1正则化
```

```
ls=Lasso(alpha=1)
```

```
ls.fit(x_train,y_train)
```

```
Lasso(alpha=1, copy_X=True, fit_intercept=True, max_iter=1000, normalize=False,  
      positive=False, precompute=False, random_state=None, selection='cyclic',  
      tol=0.0001, warm_start=False)
```

```
# L1正则化后的MES
```

```
#评估
```

```
y_pred=ls.predict(x_test)
```

```
mean_squared_error(y_test,y_pred)
```

```
342.2799716114738
```

小结

本节我们主要学习了**网格搜索**和**正则化**等模型优化技术：

- **网格搜索**：使用**交叉验证**在给定的有限集合中进行**穷举搜索**并返回效果**最好**的一组参数
- **正则化**：防止过拟合
 - L1：w**绝对值**的和
 - L2：w的**平方**和
 - 弹性网络：L1和L2的组合

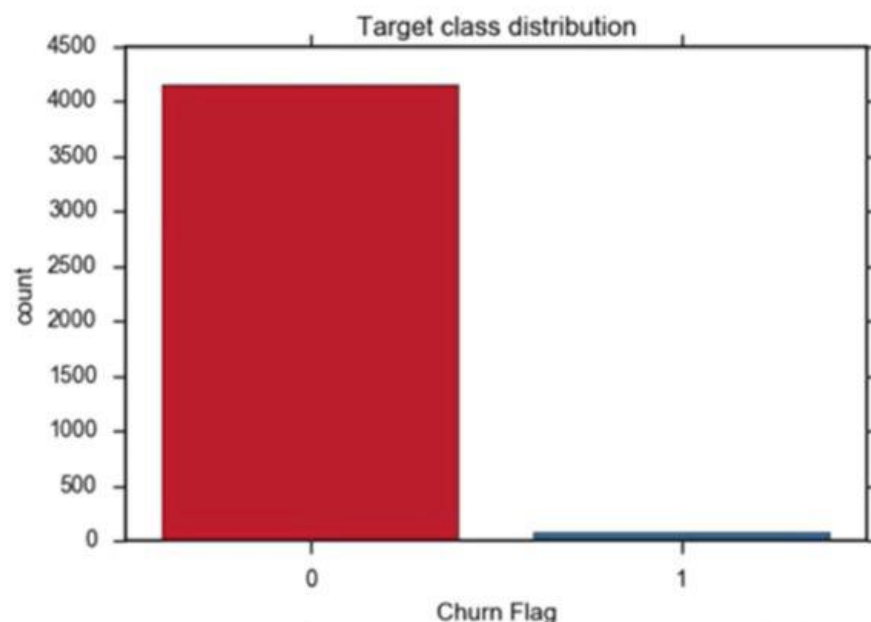


1
3
chapter

不平衡学习

什么是样本类别分布不平衡？

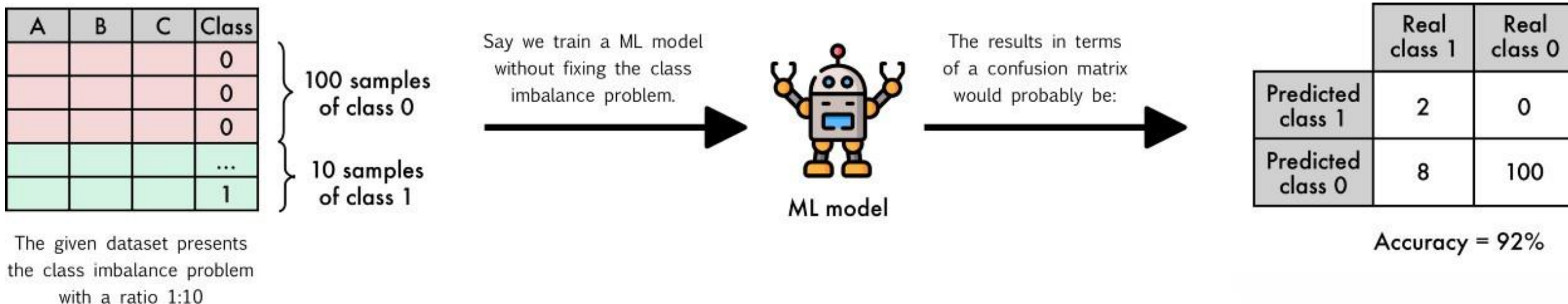
- 样本类别分布不均衡主要出现在与分类相关的建模问题上，不均衡指的是不同类别的样本量差异非常大。
- 样本分布不均衡将导致样本量少的分类所包含的特征过少，建立的模型容易过拟合。
- 如果不同分类间的样本量差异超过10倍就需要引起警觉，超过20倍就一定要想法解决了。



- 电信客户流失预测 0: 忠诚客户 1: 流失客户

准确率达 92% 的模型不能用？

- 准确率 Accuracy 常用作事实度量标准，但是对于类不平衡问题，由于准确率可能会产生误导，因此不是一个好的选择，该问题被称为准确率悖论。



准确率高达 92% 的模型不能用？

	Real class 1	Real class 0
Predicted class 1	2	0
Predicted class 0	8	100

True Positives = 2

False Positives = 0

True Negatives = 100

False Negatives = 8

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total \# samples}}$$

$$\text{Accuracy} = \frac{2 + 100}{110}$$

$$\text{Accuracy} = 0.92$$

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

$$\text{Precision} = \frac{2}{2 + 0}$$

$$\text{Precision} = 1$$

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

$$\text{Recall} = \frac{2}{2 + 8}$$

$$\text{Recall} = 0.2$$

$$\text{F1-Score} = \frac{2 \times (\text{Recall} \times \text{Precision})}{\text{Recall} + \text{Precision}}$$

$$\text{F1-Score} = \frac{2 \times (0.2 \times 1)}{0.2 + 1} = \frac{0.4}{1.2}$$

$$\text{F1-Score} = 0.33$$

样本不平衡的业务场景

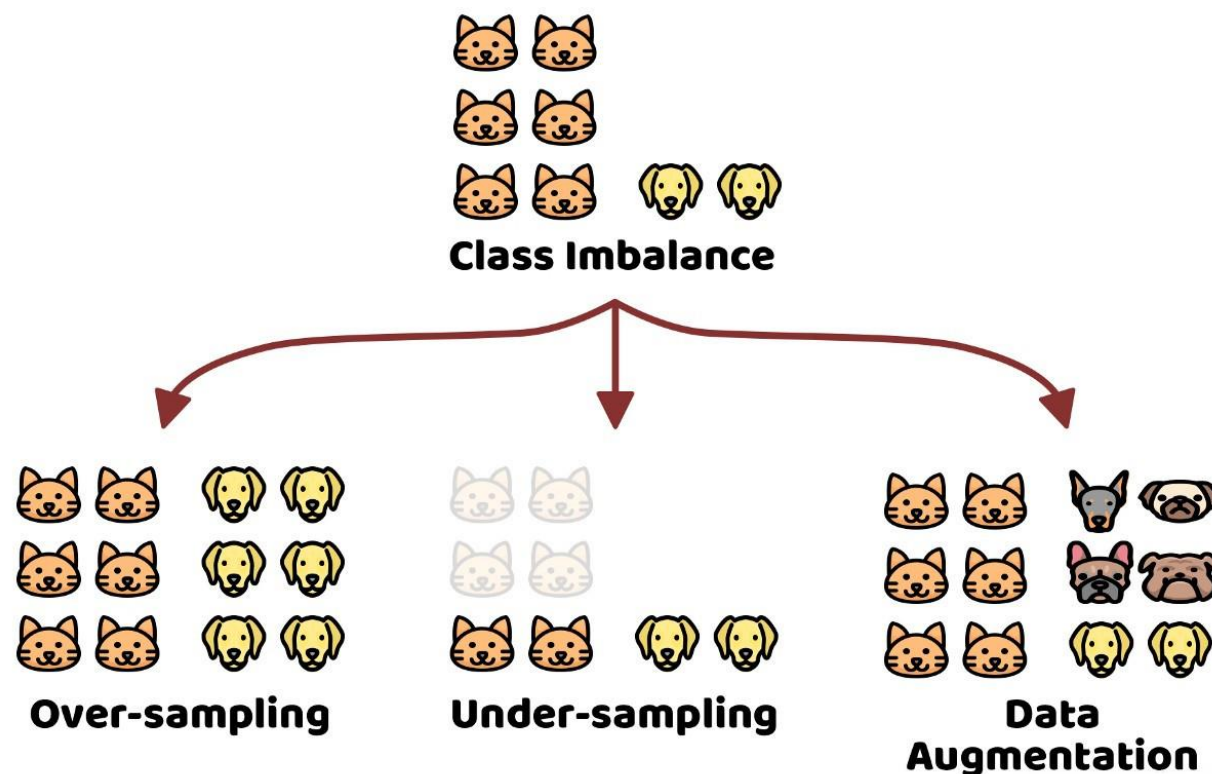
- **异常检测**：大多数企业中的异常个案都是少量的，比如恶意刷单、黄牛订单、信用卡欺诈、电力窃电、设备故障等；
- **客户流失**：大型企业的流失客户相对于整体客户通常是少量的，尤其对于具有垄断地位的行业巨擘，例如电信、石油、网络运营商等更是如此；
- **偶发事件**：罕见事件与异常检测类似，都属于发生个案较少的情况；例如，由于某网络大V无意中转发了企业的一条趣味广告，导致用户流量明显提升便属于此类；
- **低频事件**：这种事件是预期或计划性事件，但是发生频率非常低。例如，每年一次的“双11”购物节一般都会产生较高的销售额。

如何处理样本不平衡问题

- **抽样**: 过抽样和欠抽样
- **类别加权**: 少类样本损失加权
- **集成学习**: EasyEnsemble
- 选择对不平衡数据不敏感的算法: **SVM, 随机森林**

过抽样和欠抽样

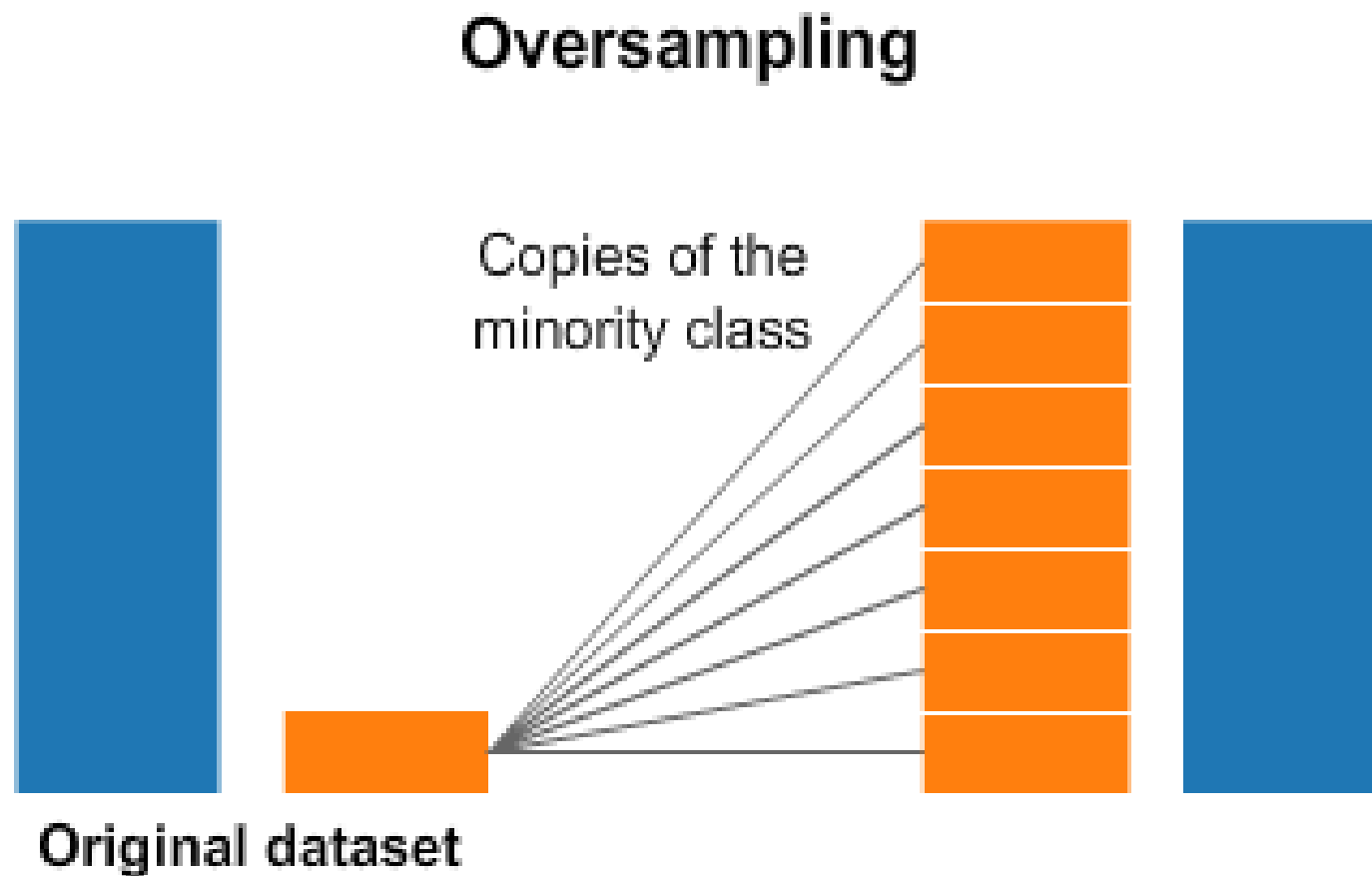
- **过抽样**: 又称上采样 (over-sampling), 其通过增加分类中少数类样本的数量来实现样本均衡, 最直接的方法是简单复制少数类样本以形成多条记录。
- **欠抽样**: 又称下采样 (under-sampling), 其通过减少分类中多数类样本的数量来实现样本均衡, 最直接的方法是随机去掉一些多数类样本来减小多数类的规模。



过抽样 (Over sampling)

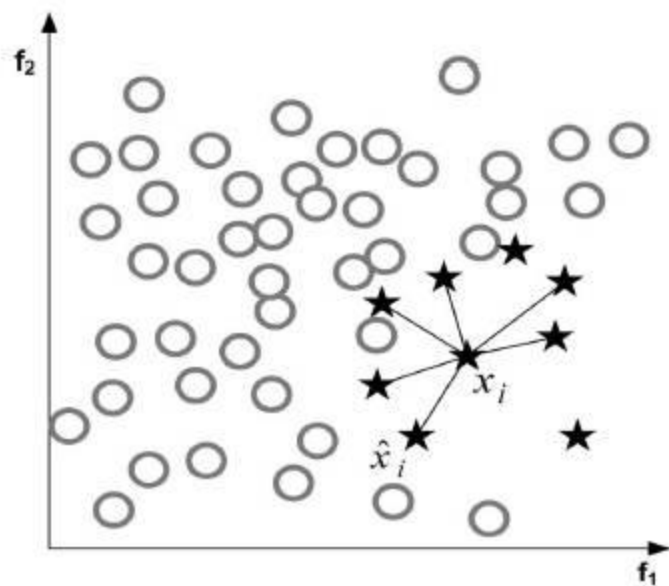
把少类样本变多

- 简单复制：容易导致过拟合
- SMOTE 算法
- 数据增强

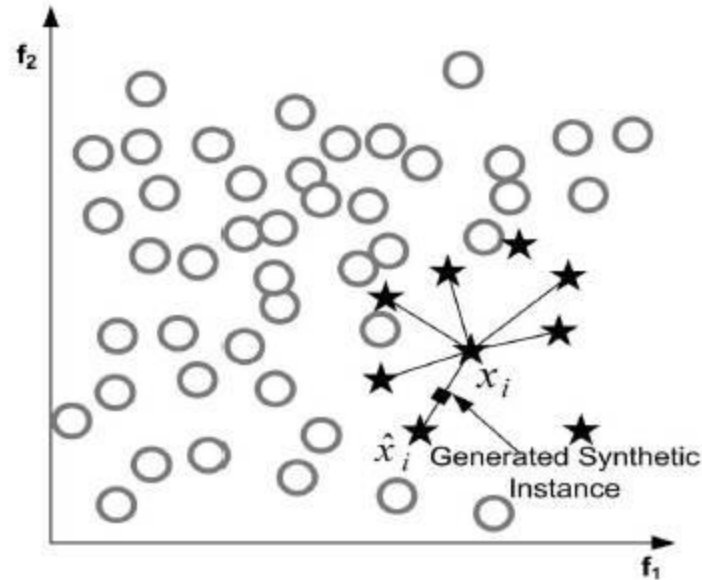


SMOTE 算法

SMOTE (synthetic minority oversampling technique) 的思想概括起来就是在**少数类样本之间进行插值**来产生额外的样本。



(a)



(b)

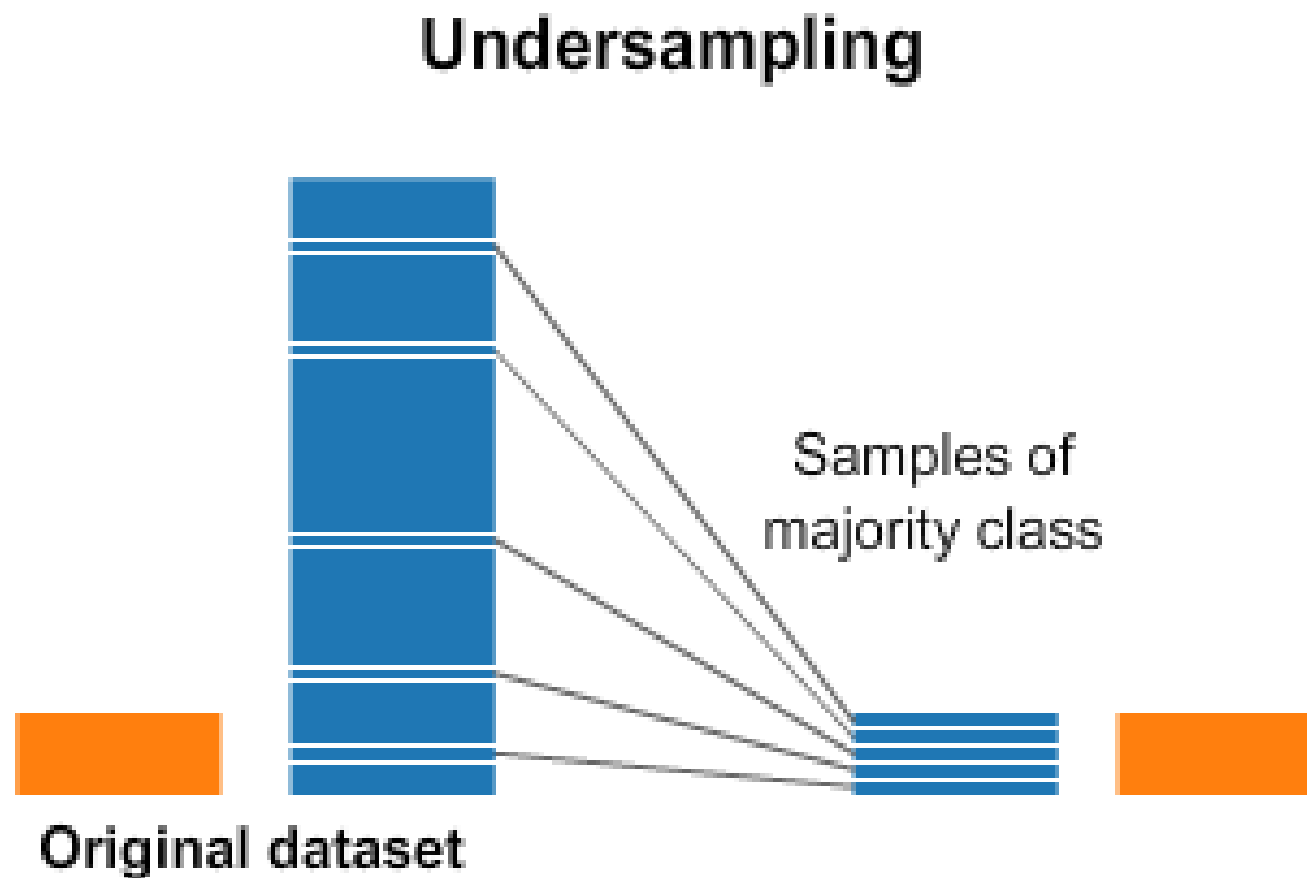
SMOTE 算法摒弃了随机过采样复制样本的做法，可以防止随机过采样易过拟合的问题，实践证明此方法可以提高分类器的性能。

思考：smote 算法有什么缺点？如何改进？

欠抽样 (Under sampling)

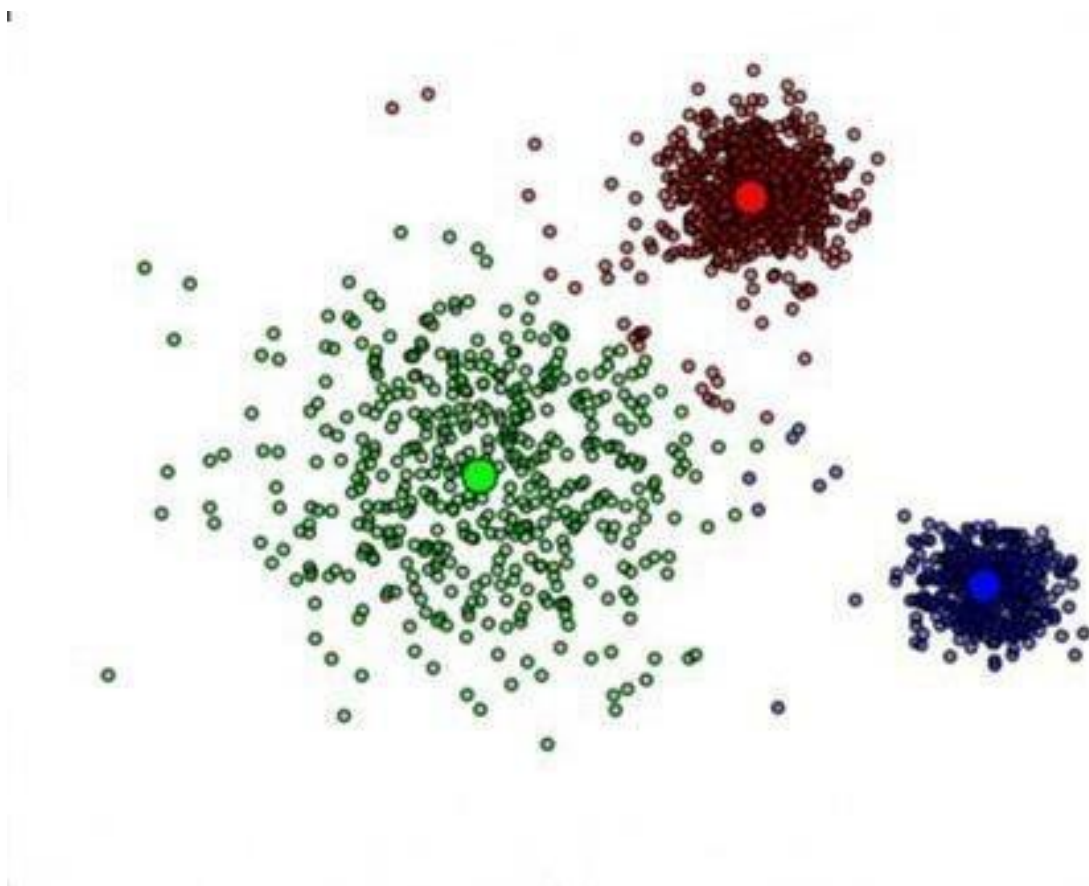
把多类样本变少

- 简单随机抽样
- 聚类下采样



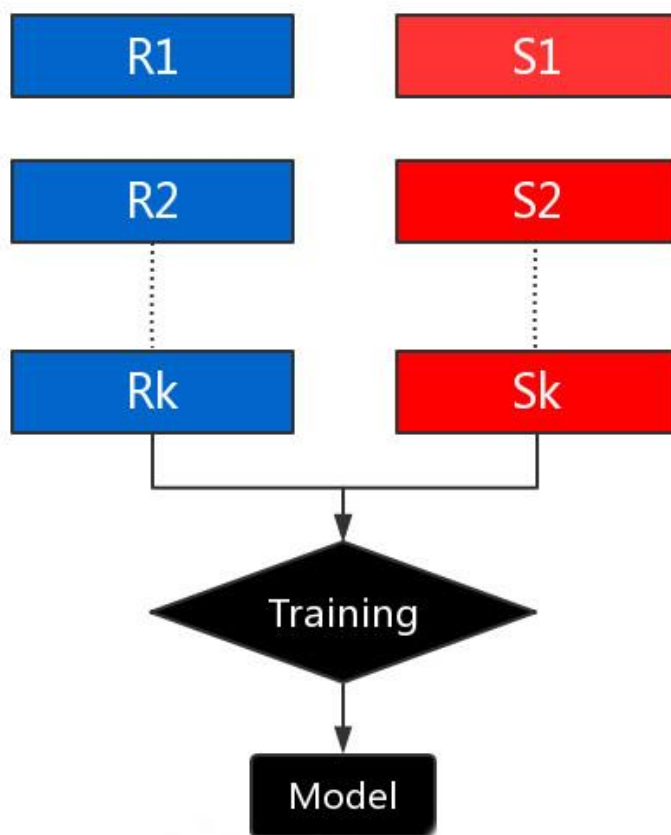
聚类下采样：对丰富类进行聚类操作

假设我们使用的方法是 K-Means 聚类算法。此时，我们可以选择 **K 值为稀有类中的数据样本的个数**，并将聚类后的中心点以及相应的聚类中心当做富类样本的代表样例，类标与富类类标一致。



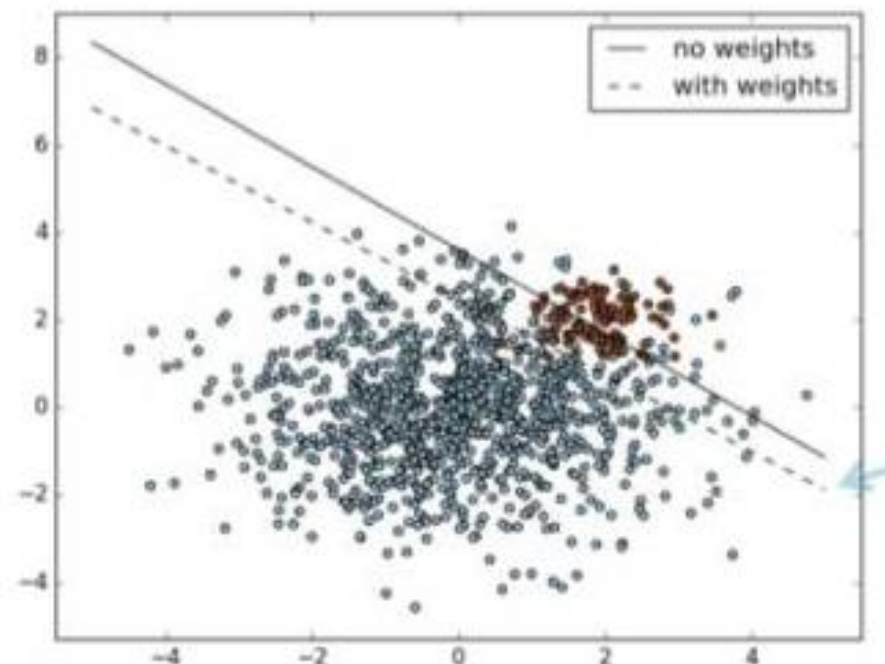
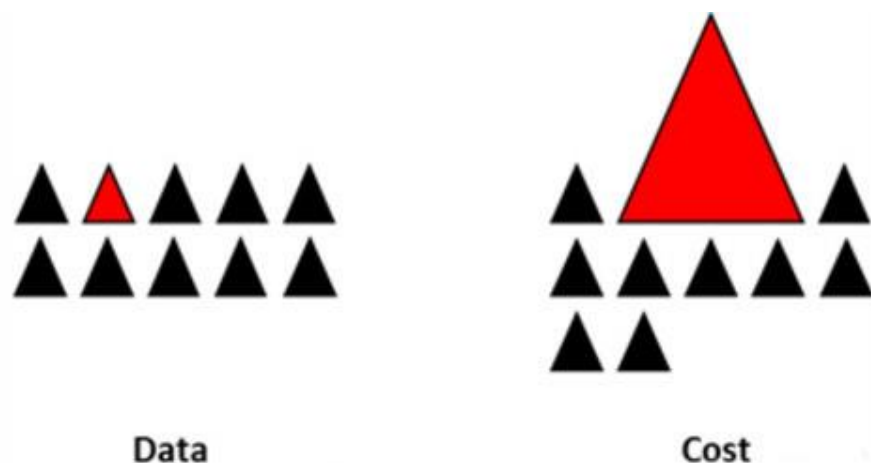
聚类下采样：聚类后的样本进行有监督学习

经过上一步骤的聚类操作，我们对富类训练样本进行了筛选，接下来我们就可以将相等样本数的 K 个正负样本进行有监督训练。



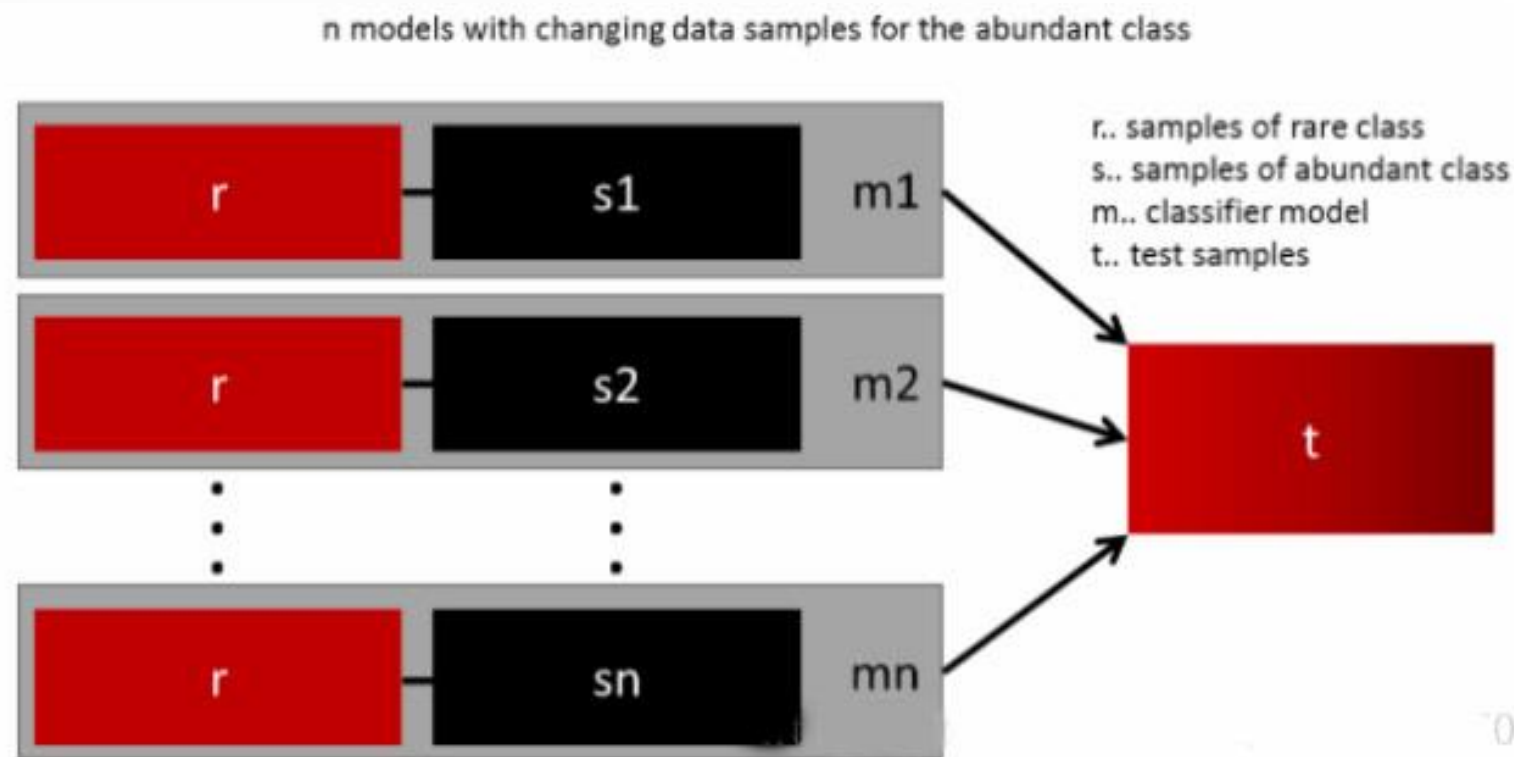
类别加权

- **类加权(class reweight)**，即在分类器训练期间直接考虑成本误差的不对称性，这使每个类的输出概率都嵌入成本误差信息。
- 效果等价于**简单随机过抽样和欠抽样**，直接在类比例中引入成本信息。



集成组合学习

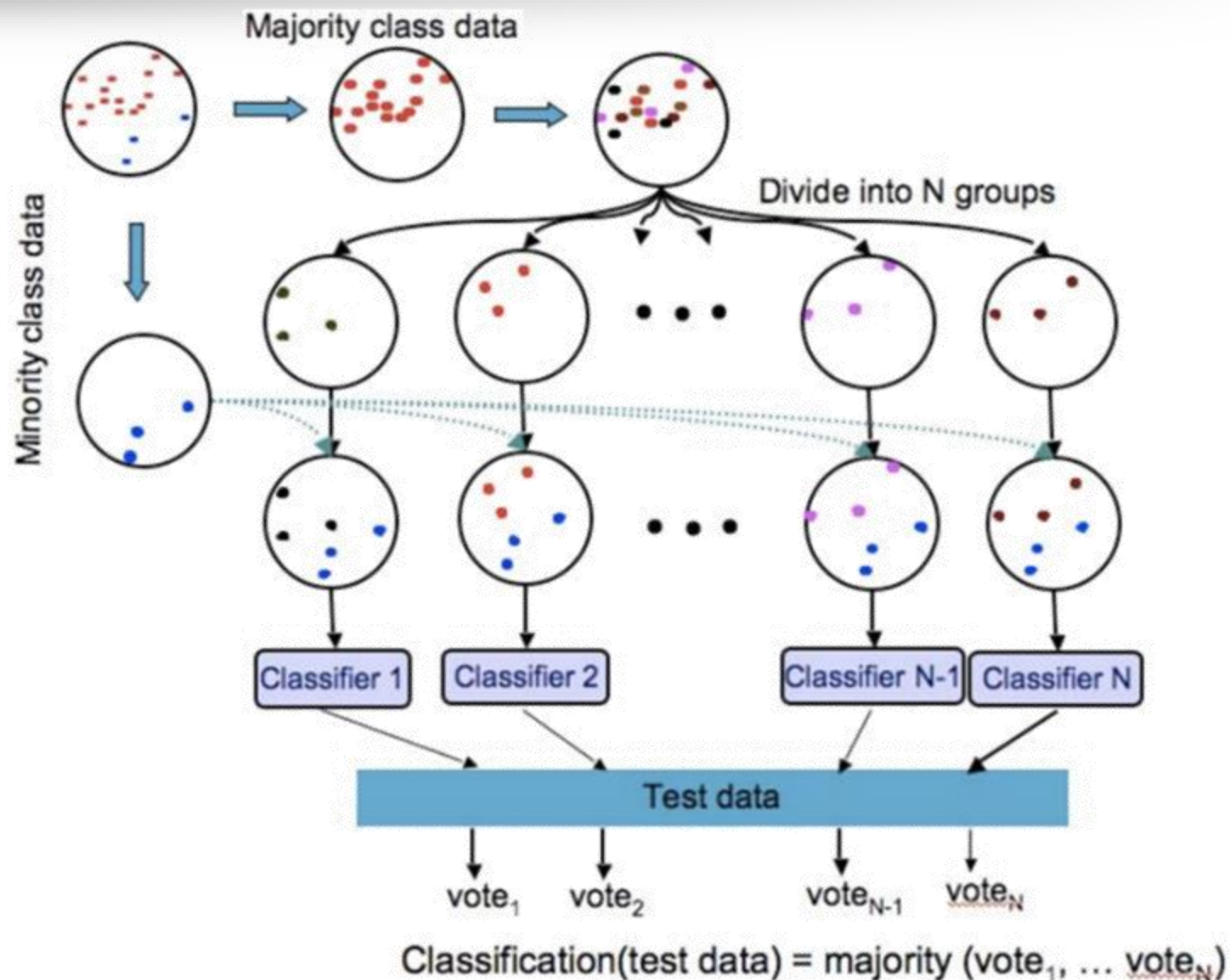
- EasyEnsemble 算法
- 用不同比例重新采样
- Bootstrap 采样



一个简单的最佳实践是建立 n 个模型，每个模型使用稀有类别的所有样本和丰富类别的 n 个不同样本。假设想要合并10个模型，那么将保留例如1000例稀有类别，并随机抽取10000例丰富类别。然后，只需将10000个案例分成10块，并训练10个不同的模型。

EasyEnsemble

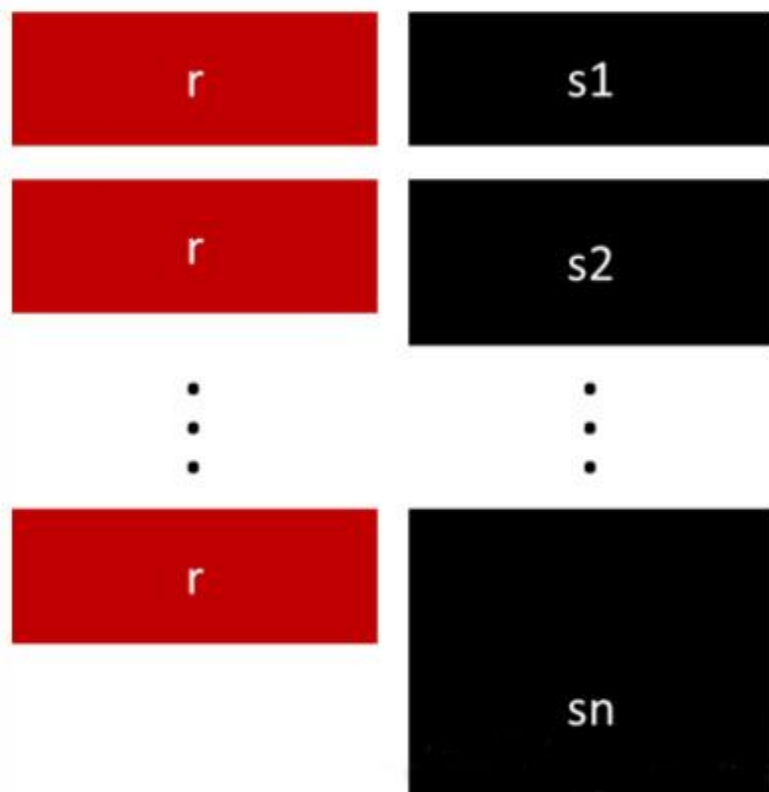
- 首先通过从多数类中独立随机抽取若干子集
- 将每个子集与少数类数据联合起来训练生成多个基分类器
- 最终将这些基分类器组合形成一个集成学习系统。



用不同比例重新采样

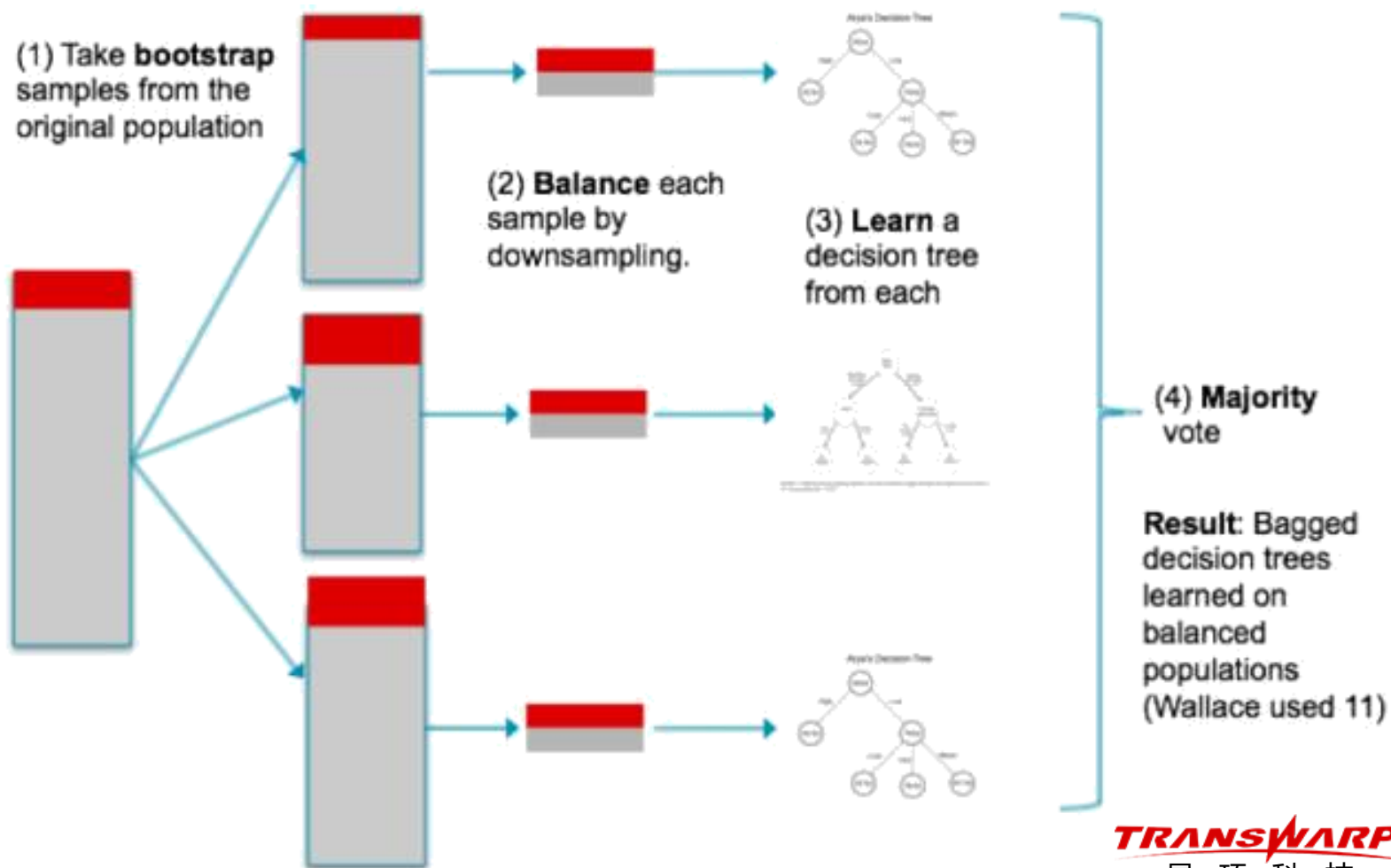
- 类似EasyEnsemble算法
- 每个子集类别比例不固定1: 1（稀有: 丰富），可以是1: 3，甚至于2: 1

n models with changing ratio between rare and abundant class



Bootstrap 采样

- 先使用 bootstrap 对原始数据进行抽样得到子集
- 对子集进行下采样平衡操作
- 训练多个模型进行融合



加载数据

```
# 导入库
import pandas as pd
from sklearn.model_selection import train_test_split # 数据分区库
from sklearn.metrics import accuracy_score, auc, confusion_matrix, f1_score, \
    precision_score, recall_score, roc_curve # 导入指标库
from imblearn.over_sampling import SMOTE # 过抽样处理库SMOTE
import matplotlib.pyplot as plt
# 读取准备
raw_data = pd.read_csv('data/Bcard.txt', delimiter=',') # 读取数据文件
raw_data.head()
```

	obs_mth	bad_ind	uid	td_score	jxl_score	mj_score	rh_score	ztc_score	zcx_score	p
0	2018-10-31	0.0	A10000005	0.675349	0.144072	0.186899	0.483640	0.928328	0.369644	
1	2018-07-31	0.0	A10000002	0.825269	0.398688	0.139396	0.843725	0.605194	0.406122	
2	2018-09-30	0.0	A10000011	0.315406	0.629745	0.535854	0.197392	0.614416	0.320731	
3	2018-07-31	0.0	A10000481	0.002386	0.609360	0.366081	0.342243	0.870006	0.288692	
4	2018-07-31	0.0	A10000069	0.406310	0.405352	0.783015	0.563953	0.715454	0.512554	

加载数据

#分割X和y

```
X,y = raw_data.iloc[:, 3:-1],raw_data.iloc[:, 1] # 分割X,y
```

#查看样本类别比例

```
y.value_counts(1)|
```

```
0.0    0.981233
```

```
1.0    0.018767
```

```
Name: bad_ind, dtype: float64
```

#数据分割

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.3, random_state=0)
```

定义输出核心评估指标的函数

#定义输出核心评估指标的函数

```
def report(model,X_test,y_test):  
    y_score = model.predict_proba(X_test) # 获得决策树的预测概率  
    y_pred=model.predict(X_test)  
    fpr, tpr, _ = roc_curve(y_test, y_score[:, 1]) # ROC  
    auc_s = auc(fpr, tpr) # AUC  
    scores = [round(i(y_test, y_pred),3 )for i in (accuracy_score,precision_score,recall_score,f1_score)]  
    scores.insert(0, auc_s)  
    # tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel() # 获得混淆矩阵  
    con_matrix=pd.DataFrame(confusion_matrix(y_test, y_pred),columns=['prediction-0','prediction-1'],index=['actual-0','actual-1'])  
    return con_matrix,pd.DataFrame([scores],columns=['auc', 'accuracy', 'precision', 'recall', 'f1'])
```

建立逻辑回归模型

```
from sklearn.linear_model import LogisticRegression
lr=LogisticRegression()
lr.fit(X_train,y_train)
report(lr,X_test,y_test)
```

```
(
      prediction-0 prediction-1
actual-0      28179          20
actual-1       535           8,
      auc accuracy precision recall    f1
0 0.779195    0.981    0.286  0.015 0.028)
```

SMOTE上采样

```
# 建立SMOTE模型对象
model_smote = SMOTE()
# 输入数据并作过抽样处理
balanced_train_X, balanced_train_y = model_smote.fit_resample(X_train, y_train)
# 观察样本是否平衡
balanced_train_y.value_counts()
```

```
0.0    65809
1.0    65809
Name: bad_ind, dtype: int64
```

```
lr3=LogisticRegression()
lr3.fit(balanced_train_X, balanced_train_y)
report(lr3, X_test, y_test)
```

```
(
      prediction-0  prediction-1
actual-0         23089         5110
actual-1          211         332,
      auc  accuracy  precision  recall    f1
0  0.788308    0.815     0.061   0.611  0.111)
```

聚类下采样

```
from imblearn.under_sampling import ClusterCentroids
cc = ClusterCentroids(random_state=0)
cluster_train_X, cluster_train_y = cc.fit_resample(X_train, y_train)
cluster_train_y.value_counts()
```

```
0.0    1255
```

```
1.0    1255
```

```
Name: bad_ind, dtype: int64
```

```
lr4=LogisticRegression()
```

```
lr4.fit(cluster_train_X, cluster_train_y)
```

```
report(lr4, X_test, y_test)
```

```
(
      prediction-0  prediction-1
actual-0      18254      9945
actual-1       154      389,
      auc  accuracy  precision  recall    f1
0  0.767481    0.649    0.038    0.716  0.072)
```

类别加权

#设置类别权重平衡

```
lr2=LogisticRegression(class_weight='balanced')  
lr2.fit(X_train, y_train)  
report(lr2, X_test, y_test)
```

```
(          prediction-0  prediction-1  
actual-0          23010          5189  
actual-1           209           334,  
          auc  accuracy  precision  recall    f1  
0  0.788945    0.812      0.06    0.615  0.11)
```

EasyEnsemble

```
from imblearn.ensemble import BalancedBaggingClassifier
bbc = BalancedBaggingClassifier(base_estimator=LogisticRegression(),
                                replacement=False,
                                random_state=0)

bbc.fit(X_train, y_train)
report(bbc, X_test, y_test)
```

```
(
      prediction-0  prediction-1
actual-0          21769          6430
actual-1           191          352,
      auc  accuracy  precision  recall    f1
0  0.777875    0.77    0.052    0.648  0.096)
```



Q&A

TRANSWARP
星环科技